

FamPlan — Lab 1 AI Critique Report

AI critique of the use case diagram and requirements · SC2006 Software Engineering · Team
Banana-Five-O

AY2026 Semester 1

[About this report](#)

[Part 1 — Use Case Diagram Critique](#)

[Use Case Diagram Critique](#)

[Part 2 — SRS Review against IEEE 830-1998](#)

[SRS Review — IEEE 830-1998](#)

[Part 3 — Use Case Descriptions Critique](#)

[Use Case Descriptions Critique](#)

[Appendix A — Traceability Matrix](#)

[Traceability Matrix — Requirements ↔ Use Cases ↔ Diagram](#)

[Appendix B — Recommended Canonical Glossary](#)

[FamPlan — Recommended Canonical Glossary](#)

About this report

This report records the AI's critique of the FamPlan Lab 1 use case diagram and requirements.

The documents submitted to the AI for critique are the documents submitted as Lab 1 deliverables in the companion document *FamPlan — Lab 1 Deliverables*: the functional and non-functional requirements, the data dictionary, the use case diagram, and the use case descriptions. The requirements, data dictionary and use case descriptions are unchanged since the critique.

The **use case diagram was redrawn as v4 after this critique**, acting on its findings, so Part 1 below reads against the earlier version of the diagram and the deliverables document shows the revised one.

Section	Critique	Artifact critiqued
Part 1	Use case diagram critique	use case diagram, pre-v4 version
Part 2	SRS review against IEEE 830-1998	requirements + use cases + data dictionary + diagram
Part 3	Use case descriptions critique	<code>Use Case Descriptions.md</code>
Appendix A	Traceability matrix (requirements ↔ use cases ↔ diagram)	supporting evidence for Part 2
Appendix B	Recommended canonical glossary	supporting output of the review

Part 1 — Use Case Diagram Critique

Use Case Diagram Critique

Artifact under review: Lab1/diagrams/FamPlan_Use_Case_Diagram.drawio.png **Judged against:** Functional and Non-Functional Requirements.md, Use Case Descriptions.md, Data Dictionary.md
Method: fresh read of the diagram; no reuse of any earlier critique.

1. Summary

The diagram is **structurally complete and well-organised** — all 20 use cases present, one package per functional-requirement group, four actors matching §4, and an actor generalization hierarchy that mirrors the “everything the previous role can do, plus...” layering. Its weaknesses are all in **relationship semantics and legibility**: the association lines are badly tangled, the «include»/«extend» stereotypes are garbled or missing, one use case is drawn at the wrong abstraction level, and the system’s defining external dependencies (§5) are invisible.

Overall: a strong first-pass structure that would mislead a reader on *how the pieces relate*, not on *what the pieces are*.

2. What the diagram gets right

Dimension	Assessment
Use case coverage	All 20 use cases from Use Case Descriptions.md appear, correctly numbered (1.1–8.3). Nothing dropped, nothing invented.
Package structure	Eight packages map 1:1 onto the eight §9 functional-requirement groups. Grouping is a genuine aid to tracing diagram → requirements.
Actor roster	Visitor, Registered User, Data Administrator, System (scheduled process) — exactly the four actors of §4.
Actor generalization	Visitor ← Registered User ← Data Administrator is modelled with generalization, which correctly expresses §4’s additive privilege model and resolves the descriptions’ abstract “User” actor onto a concrete base actor.
Scheduled process	System → Run Scheduled Daily Sync (8.3) faithfully models FR-ADM-04’s no-user-present trigger.

3. Findings (most significant first)

F1 — Association lines are unreadable, which defeats the diagram’s purpose

The primary job of a use case diagram is to make the **actor** → **goal** mapping obvious at a glance. Here the associations cross so heavily across the centre of the canvas that a reader cannot confirm, by inspection,

which actor reaches which use case. A diagram whose central claim (who can do what) cannot be verified visually has failed its main function, however complete its contents. **Fix:** route edges orthogonally, colour edges per actor, place each actor next to the cluster it uses, and draw each shared goal from the **base** actor only.

F2 — «include» / «extend» relationships are garbled or missing

Use Case Descriptions.md specifies four relationships precisely:

Relationship	Type	Source
4.1 View Flat Profile Details → 6.1 Calculate Suitability Score	include	UC 4.1 Includes
6.2 Compare Flat Profiles → 6.1 Calculate Suitability Score	include	UC 6.2 Includes
8.2 Trigger Manual Source Refresh → 8.1 View Data Source Sync Status	include	UC 8.2 Includes
3.2 Apply Saved Preferences → 3.1 Search and Filter Flats	extend	UC 3.1 flow, FR-SRCH-03

In the diagram the three includes appear as dashed connectors whose stereotype labels render as <> / <<>> rather than «include», and arrow direction is not legible. The **3.2 → 3.1 extend is absent** — 3.2 reads as just another actor-associated use case, silently dropping a relationship the spec defines. **Fix:** label every dashed connector explicitly «include» or «extend»; point include arrows to the included use case and extend arrows to the base; add the missing 3.2→3.1 extend.

F3 — External data providers are not modelled, hiding the system’s defining dependency

§5 and Assumptions A-1...A-5 make five public data sources (data.gov.sg ×3, OneMap, LTA DataMall) central, and use cases 2.1, 4.2, 5.1, 5.2, 8.2, 8.3 are built on them. The diagram shows no external/secondary actors, so a reader would conclude the system is self-contained — the single biggest architectural risk (external dependency) is invisible. **Fix:** add supporting actors (OneMap, data.gov.sg, LTA DataMall) on the right, associated to the live-dependent and ingestion use cases. (Local-query use cases 3.1/4.1 correctly stay unwired, per A-1 / NFR-PERF-01’s exclusion of external calls.)

F4 — Calculate Suitability Score (6.1) is drawn at the wrong abstraction level

UC 6.1’s own description reads “Includes: None ... included by 4.1 and 6.2.” It is a **sub-function** invoked by other use cases, not a standalone user goal. If the diagram gives it a direct primary-actor association (it appears to), it both conflates goal levels and duplicates the include relationship. **Fix:** model 6.1 as an «include» target only, with no primary-actor line, and de-emphasise it visually.

F5 — Notation needs verification against UML conventions

- **Generalization arrowheads** must be *hollow triangles on the parent end* (pointing to Visitor; pointing to Registered User from Data Administrator). Confirm direction — a reversed triangle inverts the privilege model.
- The garbled stereotype text (F2) suggests the stereotype was typed as literal angle brackets rather than guillemets; use «...».

F6 — Actor–association correctness cannot be confirmed, and §4 vs Data Dictionary disagree on “User”

Because of F1, the following can’t be verified from the render and **must** be checked, especially as the spec itself is internally inconsistent here (see the SRS review, C1): the requirements §4 and Use Case Descriptions say visitors *can* search, view profiles/market, explore location, compare, and run the HFE pre-check, but `Data Dictionary.md` §8 says Registered User “is the actor behind every FR whose Actor column reads ... ‘User’”, which would exclude visitors. **The diagram is the tie-breaker**: if 3.1, 4.1, 4.2, 5.1, 5.2, 6.2 and 7.1 attach to **Visitor** (base actor), the diagram endorses §4 and contradicts the Data Dictionary — a discrepancy to reconcile in the text, not just the drawing. **Verify**: 1.1 attaches to Visitor only; 1.2–1.5, 2.1, 3.2, 6.3 to Registered User; 8.1, 8.2 to Data Administrator; 8.3 to System; and every public use case attaches to Visitor.

4. Minor notes

- **Actor naming vs FR-ADM-04.** The diagram’s “System (scheduled process)” is the actor for 8.3; FR-ADM-04’s Actor column says “System clock”. Harmonise the label across artifacts.
 - **Package 8 title** (“... and System Resilience”) implies resilience behaviour (FR-RES-02/03) that has no use case. Acceptable (resilience is cross-cutting), but the title slightly over-promises what the diagram shows.
 - **Redundant associations.** If both Visitor and Registered User connect to a shared use case, the second is redundant under generalization — draw shared goals from Visitor once.
-

5. Fix checklist

- ☐ De-tangle: orthogonal routing, per-actor colours, base-actor-only shared associations.
- ☐ Label and direct all «include» edges; add the missing 3.2→3.1 «extend».
- ☐ Add external supporting actors for §5 providers.
- ☐ Demote 6.1 to an include-only sub-function.
- ☐ Verify generalization arrowheads point to the parent.
- ☐ Confirm every public use case reaches Visitor; reconcile the §4 vs Data-Dictionary “User” conflict in the text.

(A corrected source implementing most of these already exists at `Lab1/diagrams/FamPlan_Use_Case_Diagram_corrected.drawio`.)

Part 2 — SRS Review against IEEE 830-1998

SRS Review — IEEE 830-1998

Specification under review (graded as one distributed SRS): Functional and Non-Functional Requirements.md + Use Case Descriptions.md + Data Dictionary.md + diagrams/FamPlan_Use_Case_Diagram.drawio.png **Standard:** IEEE Std 830-1998, *Recommended Practice for Software Requirements Specifications* — §4.3 quality characteristics of a good SRS.

1. Scorecard

Each of IEEE 830 §4.3's eight characteristics, rated **Strong** / **Adequate** / **Weak**.

#	Characteristic	Rating	One-line basis
a	Correct	Strong	Requirements are grounded in real HDB/data.gov.sg/OneMap behaviour; illustrative parts (HFE ruleset) are explicitly flagged as such (A-6).
b	Unambiguous	Adequate	Atomic requirements with Input/Output/Verification columns are excellent, but the “Must” convention is self-contradictory and actor terms diverge across artifacts.
c	Complete	Adequate	FR/NFR/data/assumptions/scope coverage is thorough; several quantitative limits are left as unfilled TBDs and classic 830 interface/overall-description sections are absent.
d	Consistent	Adequate	Cross-references are unusually disciplined, but one substantive contradiction (“User” actor scope) and one naming drift (Visitor vs Guest) remain.
e	Ranked for importance and stability	Weak	Importance is present (Must/Should priority); stability/volatility ranking is entirely absent , though volatile items clearly exist.
f	Verifiable	Strong	Every FR carries concrete verification cases; NFRs are quantified (percentiles, counts, thresholds). A model strength.
g	Modifiable	Strong	Modular sections, stable IDs, config isolation (NFR-MAINT-01); one deliberate duplication (assumptions copied into the data dictionary) is a small drift risk.
h	Traceable	Strong	FR IDs are cited from data-dictionary entities and assumptions; verification gives forward traceability. Only a consolidated matrix is missing (now supplied).

Headline: an unusually rigorous, verifiable, traceable specification whose weak point is **ranking (no stability dimension)**, with fixable ambiguity/consistency defects concentrated in the actor vocabulary and the modal-verb convention.

2. Detailed findings

Findings are keyed by the characteristic they most affect. Each has a concrete fix.

Unambiguous (b)

B1 — The “Must” convention is defined twice, with two different meanings. §8 *Requirement Conventions* states both: - “**Must** states a mandatory constraint.” (a modal verb, like Shall) - “**Must** means essential for the agreed minimum viable product; **Should** means important but negotiable...” (a **priority** level)

The Priority column then uses Must/Should as priority. So “Must” is overloaded between *constraint modality* and *priority rank*, and a reader cannot tell which sense applies where. **Fix:** separate the two axes. Use *Shall* for capabilities and *Must* for constraints in requirement **text**; rename the Priority column values to **Essential / Desirable** (or High/Low) so priority never reuses a modal verb.

B2 — The unauthenticated actor has three names. “Visitor” (requirements §4, diagram) vs “Guest / Applicant” (Data Dictionary §8). Same concept, inconsistent label — an ambiguity under 830 §4.3.2, which asks that each term have one meaning and one name. **Fix:** pick one canonical term (recommend **Visitor**) and use it everywhere. Captured in `review/CONTEXT.md`.

B3 — Soft thresholds referenced but not yet valued. “insufficient data (below configured threshold)” (FR-MKT-08 / UC 4.2), “documented rounding” (FR-DEC-02), note “length boundary” (FR-DEC-07), shortlist “documented limit” (§4). These are referenced as if defined but their values are not in the spec. **Fix:** give each a concrete value or a single “Configurable parameters” table, so verification cases are executable.

Complete (c)

C1 — Unquantified limits (TBDs) are present but not marked or tracked. IEEE 830 §4.3.3 says an SRS is complete only if TBDs are labelled and a resolution process exists. The spec has genuine TBDs — shortlist size limit, note length, rounding rule, editable-field list for FR-ACC-05 (Data Dictionary explicitly defers it) — but they are scattered in prose rather than flagged as TBD. **Fix:** add a short **Open Items / TBD** register listing each, its owner, and its resolution deadline.

C2 — Classic 830 structural sections are missing. Judged as a whole the set is content-complete, but it lacks 830’s *Overall Description* (product perspective, product functions summary, user characteristics as a first-class subsection, operating environment, design/implementation constraints) and an explicit *External Interface Requirements* section (user/hardware/software/communications interfaces). UI mockups exist under `ui_mockups/` but are not referenced from the spec as the UI-interface requirement. **Fix:** add a brief §Overall Description and an §External Interfaces that points at the mockups and names the five provider APIs as software interfaces.

C3 — Two functional requirements have no home use case (acceptable, but note it). FR-RES-02 (fault isolation) and FR-RES-03 (stale-data warning) trace to no use case — they are cross-cutting behaviours. This is defensible, but 830 completeness prefers every requirement be reachable; state explicitly that these are cross-cutting and verified by the NFR-REL/NFR-DATA tests.

Consistent (d)

C4 — “User” actor scope is internally contradictory (the most important consistency defect). - `Functional and Non-Functional Requirements.md` §4 and `Use Case Descriptions.md`: “**User (Visitor or Registered User)**” — visitors are included in every `Actor = User` requirement (search, market, location, scoring, compare, HFE). - `Data Dictionary.md` §8: “**Registered User ... the actor behind every FR whose**

Actor column reads ‘Authenticated user’ or ‘User’” — which **excludes visitors** from those same requirements.

These cannot both be true. It changes whether an unauthenticated visitor may run FR-SRCH/FR-MKT/FR-LOC/FR-DEC(scoring, compare)/FR-HFE at all. **Fix:** adopt §4’s meaning (visitors included), and correct the Data Dictionary §8 definition of Registered User to “the actor behind every FR whose Actor column reads ‘Authenticated user’; requirements marked ‘User’ are available to Visitors too.” The diagram’s Visitor associations should match.

C5 — Duplicated assumptions table can drift. Assumptions A-1...C-4 are maintained in the requirements doc **and** copied into the Data Dictionary. The Data Dictionary names the requirements doc authoritative, which is good practice, but a duplicated table is a live inconsistency risk. **Fix:** keep one copy; have the Data Dictionary link to it rather than reproduce it (this also improves *modifiability*, g).

Ranked for importance and stability (e) — weakest characteristic

E1 — No stability/volatility ranking anywhere. 830 §4.3.5 asks each requirement be ranked on **two** axes: importance *and* stability (expected likelihood of change). The spec ranks importance (Priority = Must/Should) but never stability — even though volatile items are obvious: the HFE ruleset is illustrative and will change (A-6); provider APIs and tokens may change (A-2); soft thresholds are unset (B3). **Fix:** add a **Stability** column (Stable / Volatile) or tag volatile requirements. At minimum, flag every A-6/A-2-dependent requirement as Volatile.

E2 — Importance is nearly monotone. Almost every FR is “Must”, which weakens ranking’s usefulness for scope negotiation. Consider whether some Musts are truly essential for MVP or are actually Desirable.

Verifiable (f) — strength, with one caveat

F1 — Verifiability depends on the unset thresholds in B3. Verification cases such as FR-MKT-08’s “insufficient data” or FR-DEC-02’s “documented rounding” cannot be executed until those values exist. Resolving B3 closes this. Otherwise verifiability is exemplary and should be preserved as the spec evolves.

Modifiable (g) / Traceable (h) — strengths

G1/H1 — Preserve what is working. Stable IDs, per-requirement verification, config isolation (NFR-MAINT-01), and FR-ID citations from the data dictionary are exactly what 830 §4.3.6–4.3.8 reward. The only additions needed are the consolidated traceability matrix (supplied in `review/Traceability_Matrix.md`) and removing the duplicated assumptions table (C5).

3. Priority of fixes

1. **C4** — resolve the “User”/visitor contradiction (correctness of scope).
2. **B1** — fix the double-defined “Must” convention (affects reading of every row).
3. **E1** — add a stability ranking (the one weak characteristic).
4. **B2 / C5** — unify actor vocabulary; de-duplicate assumptions.
5. **B3 / C1** — value the soft thresholds and open a TBD register.
6. **C2** — add Overall Description + External Interfaces sections.

Items 1–2 are small text edits with large clarity payoff; item 3 is the only structural gap against 830.

Part 3 — Use Case Descriptions Critique

Use Case Descriptions Critique

Artifact under review: Lab1/Use Case Descriptions.md (use cases 1.1–8.3) **Judged against:** use-case authoring best practice — Cockburn, *Writing Effective Use Cases*; Larman, *Applying UML and Patterns*; the UML «include»/«extend» semantics — cross-checked against Functional and Non-Functional Requirements.md. **Scope:** the *use cases themselves* as engineering artifacts — goal level, precondition rigor, scenario structure, relationship semantics, black-box discipline. This is deliberately **not** a re-run of the diagram critique (Diagram_Critique.md), the IEEE 830 SRS review (SRS_Review_IEEE830.md), or the actor-vocabulary work (CONTEXT.md); those findings are referenced, not repeated. **Method:** fresh read of every use case; each tested against a named best-practice rule.

1. Summary

These are **above-average use cases** — the template is disciplined, alternative flows are tied to step numbers, Exceptions are cleanly separated from Alternative Flows, and every use case carries success *and* failure postconditions plus explicit NFR traceability. That structural quality is real and worth preserving.

The weaknesses are not in coverage but in **use-case discipline**: a systemic misuse of the *Preconditions* field (things the use case actually validates are listed as assumptions it may not check), two use cases that **bundle more than one goal**, one use case written at the **wrong goal level**, relationship fields (Includes) that are **asymmetric and occasionally misused**, and **branching / implementation detail leaking into the main success scenario**. None is fatal; all are the kind of defect an SC2006 grader trained on Cockburn will circle.

Overall: a well-formed set whose main-flow *contents* are strong, but whose *contracts* (preconditions, relationships, goal levels) need tightening.

2. What the descriptions get right

Dimension	Assessment
Template completeness	Every use case carries ID, actor, description, pre/postconditions, priority, frequency, main flow, alternative flows, exceptions, includes, special requirements, assumptions, and notes. Few student sets are this complete.
Exceptions vs Alternative Flows	Business variations (Alternative Flows) are correctly separated from system/technical failures (Exceptions) — exactly Cockburn’s distinction. Consistently applied across all 19.
Step-anchored alternates	Alternatives are keyed to the step they diverge from (AF-S4.1, AF-S7), which is the right way to make them traceable.
Dual postconditions	Each use case states both a success guarantee and a failure guarantee — close to Cockburn’s success/minimal-guarantee pairing.
NFR traceability	Special Requirements cite the exact NFR each flow must satisfy (NFR-SEC-02, NFR-PRIV-01, NFR-PERF-01...), giving forward traceability from behaviour to quality attribute.
Abstract actor handled explicitly	User (Visitor or Registered User) is used consistently, and 6.1 even spells out “session defaults for a Visitor” — the descriptions sit on the correct side of the §4-vs-Data-Dictionary “User” contradiction (see SRS_Review_IEEE830.md C4). The use cases are not the source of that defect.

3. Findings (most significant first)

F1 — Preconditions are used for conditions the use case actually validates (systemic)

A precondition, by definition, is a state the system may **assume true and will not re-check**; if a use case can fail because the condition is false, it is not a precondition — it is an input the main or alternative flow must validate. Several use cases list as *preconditions* exactly the things they then check, and provide an **alternative flow for the precondition being false** — a direct contradiction:

Use case	Stated precondition	Contradicting flow/alternate
6.2 Compare Flat Profiles	“At least two profiles have been selected”	AF-S1: “Fewer than two ... selected → rejected” — cannot occur if the precondition holds
7.1 Perform Indicative HFE Pre-check	“User acknowledges the pre-check is general guidance”	Acknowledgement is step 1 of the main flow, and AF-S1 handles non-acknowledgement
3.2 Apply Saved Preferences	“has at least one saved preference value”	AF-S3: “No saved preferences exist → action disabled”
1.2 Login / Logout	“User holds a registered account (for login)”	AF-S3: “Unknown account → generic failure”
1.3 Reset Password	“User holds a registered account with a valid email on file”	Step 3.2 / AF-S3.2: unknown email path

Why it matters: this is the most common precondition error in use-case writing, and it is here five times, so it reads as a pattern, not a slip. It also weakens testability — a reader cannot tell what the system *guarantees* going in versus what it *enforces*. **Fix:** demote each of these to the flow. Replace the precondition with the genuine, unchecked assumption (e.g. 6.2: “The user is in the comparison feature”; 7.1: “The pre-check entry point is reachable”), and let the count/acknowledgement/existence check live as a validated step with its alternative. A precondition should never have a matching alternative flow.

F2 — Two use cases bundle more than one goal

A use case should capture **one** primary actor achieving **one** goal in one sitting (Cockburn, user-goal / “sea” level).

- **1.2 Login / Logout** fuses two distinct goals with different preconditions and postconditions into a single flow whose steps 1–4 are login and 5–7 are logout — a sequence **no actor performs in one sitting**. The requirements already split them (FR-ACC-02 authenticate, FR-ACC-03 logout), so the use case is *coarser than its own FRs*, breaking 1:1 traceability.
- **6.3 Manage Shortlist and Notes** bundles four operations — add, remove, create/edit/clear note — across **two different priorities** (“Must (shortlist); Should (notes)”). Its main flow chains add → note → remove as one scenario. Mixing a Must and a Should in one use case makes the priority field meaningless for scope negotiation and prevents the Should-notes behaviour from being dropped independently. FR-DEC-05/06/07 correctly separate them.

Fix: split 1.2 into *Log In* and *Log Out*. Either split 6.3 into *Manage Shortlist* (Must) and *Annotate Shortlisted Profile* (Should), or keep the CRUD grouping but record the two priorities per-operation and stop chaining them into one narrative scenario. (2.1 Manage Housing Preferences is also broad but remains a **single** goal — setting preferences — so it is acceptable as a CRUD-style use case.)

F3 — 6.1 Calculate Suitability Score is written at the wrong goal level

6.1’s own fields give it away: *Includes*: “None — included by 4.1 ... and 6.2”; *Frequency*: “invoked by 4.1, 6.2, and shortlist views.” It is a **sub-function** (“fish” level) — a step other use cases invoke, not a standalone goal a user sits down to achieve. Yet it is given a primary **Actor** (User), a **Priority** (Must), and a white-box main flow describing the normalisation-and-sum **algorithm** step by step. This both over-promotes it to a user goal and pulls internal computation into a use case. (The diagram critique flags the same node at F4; this is the *description-level* half of that defect.) **Fix:** keep 6.1 as an included sub-function: drop the primary-actor association, let its priority be inherited from its includers, and frame it as “the behaviour invoked by 4.1/6.2.” The itemised, user-visible explanation (contributions, weights, missing-data treatment) stays, since that is observable; the “normalise × weight, then sum” narration is internal design and can be compressed.

F4 — Relationship fields are asymmetric and partly misused

The template records **Includes** but has **no Extends / Included by / Trigger field**, so relationships land inconsistently:

- **The 3.2 → 3.1 «extend» is unrecorded.** 3.1’s flow step 3 says “Apply saved preferences ... which extends this use case,” but **both 3.2 and 3.1 have Includes: None** and neither carries the extend in a structured field. A relationship the spec relies on lives only in prose. (Diagram critique F2 reaches the same conclusion from the drawing.)

- **8.2 → 8.1 «include» is likely the wrong stereotype.** 8.2 “Trigger Manual Source Refresh” declares it *includes* 8.1 “View Data Source Sync Status,” and step 1 is “opens the source-status view.” Triggering a refresh does not *execute the whole view-status use case as a mandatory sub-step*; the status view is **navigational context / a precondition** (“source-status view is available,” which 8.2 already lists). This is the classic misuse of «include» for UI sequencing.
- **6.1’s claimed callers don’t all declare it.** 6.1 says it is “invoked by ... shortlist views,” but 6.3 declares no include of 6.1. Either 6.3 should include it or the claim should be dropped.

Fix: add explicit **Trigger**, **Extends**, and **Included by** rows to the template; record every relationship in a structured field (not prose); reclassify 8.2→8.1 as a precondition rather than an include; reconcile 6.1’s caller list with the callers’ own **Includes**.

F5 — Branching and duplication inside the main success scenario

The main success scenario should be **one typical, branch-free path**; conditionals belong in extensions, stated **once**. Several flows violate this:

- **Inline conditionals duplicated as alternates (double-spec, drift risk).** 1.3 step 3 embeds the “if email matches / if not” branch *and* repeats it as **AF-S3.2**. 4.2 step 3.1 embeds “If the comparable set is empty ...” *and* repeats it as **AF-S3**. The same rule now lives in two places and can drift.
- **Over-optional main flow.** 3.1 Search and Filter Flats has steps 2, 3, and 4 all marked “optionally,” so the *main* scenario never commits to what a typical successful search looks like — the happy path is under-defined.

Fix: write each main flow as a single concrete success path; move every conditional to an extension and state it once; give 3.1 a definite typical scenario (e.g. “user enters a town and applies a price filter”) and push the truly optional steps to extensions.

F6 — Duplicate alternative-flow identifiers break unique citation

Several use cases reuse one label for **different** alternatives, so an alternate cannot be cited or tested uniquely:

- **1.2** has three distinct **AF-S3** entries (wrong password, unknown account, logout).
- **1.3** has two **AF-S7** entries (expired token, reused token).
- **6.2** has two **AF-S1** entries (bad count, duplicate).
- **5.2** has two **AF-S1** entries (unsupported mode, invalid destination).

Fix: give every alternative a unique suffix (**AF-S3.1**, **AF-S3.2**, **AF-S3.3**) — the convention already used correctly in 1.1 (**AF-S4.1...4.4**). This directly enables one test per alternate.

F7 — Implementation (“how”) leaks into black-box behavioural steps

Use case flows should describe **observable** behaviour, not mechanism. A few steps state design:

- **1.1** step 5: “creates the account record with a **hashed password** (NFR-SEC-02).”
- **1.2** step 6: “invalidates the session **token server-side**.”
- **7.1** step 2: questionnaire “held only in **request-scoped session state** (never persisted — see C-1).”

Each mechanism is already captured where it belongs (Special Requirements / constraint C-1), so the flow can stay black-box. **Fix:** in the flow write the observable effect (“the account is created,” “the session ends,”

“answers are collected for this session only”); leave hashing, token scope, and request-scoping to the referenced NFR/constraint.

F8 — Use-case count and FR coverage need reconciling

- **Count discrepancy.** These descriptions contain **19** use cases (1.1–1.5, 2.1, 3.1–3.2, 4.1–4.2, 5.1–5.2, 6.1–6.3, 7.1, 8.1–8.3). `Diagram_Critique.md` asserts “all 20 use cases” **twice**. One artifact is wrong — verify against the diagram and correct whichever is off.
- **FRs with no owning use case.** FR-RES-02 (fault isolation) and FR-RES-03 (stale-data warning) trace to no use case. This is defensible — they are cross-cutting — but it should be **stated** in the use case set (a short “cross-cutting behaviours, verified by NFR-REL/NFR-DATA tests” note), not left as a silent gap. (Matches `SRS_Review_IEEE830.md` C3.)

Fix: reconcile the 19/20 count across the two artifacts; add a one-line note recording FR-RES-02/03 as intentionally use-case-less cross-cutting requirements.

4. Minor notes

- **No explicit Trigger field.** Cockburn treats the trigger as a first-class element; here it is implied by “step 1.” Adding it makes the scheduled use case (8.3) and the user-initiated ones uniformly readable. (Folded into F4’s fix.)
 - **Rigid use-case pipeline.** 4.1, 4.2, 5.1, 5.2 each precondition on “a profile has been **selected** (see 3.1/4.1),” so they are not independently invocable — they presume a specific prior use case. This is realistic for a UI flow, but worth noting that several use cases are steps in a chain rather than free-standing goals.
 - **Undefined thresholds block use-case-level acceptance.** 6.3’s note “length boundary” and 4.2’s “below configured threshold” are validated in-flow but never valued, so those alternatives cannot be tested as written (cross-ref `SRS_Review_IEEE830.md` B3). A use case that checks an undefined limit is not yet verifiable.
 - **6.1’s cramped relationship field.** “Includes: None — included by 4.1 ... and 6.2” overloads one field with two opposite relationships; the `Included by` row from F4 resolves this.
 - **Actor vocabulary is already settled in `CONTEXT.md`** (Visitor / Registered User / Data Administrator / System; abstract “User” = Visitor OR Registered User). The descriptions comply, so it is not re-litigated here.
-

5. Fix checklist

- ☐ **F1** Demote every “precondition” that has a matching alternative flow (6.2, 7.1, 3.2, 1.2, 1.3) into a validated step; leave only genuinely unchecked assumptions as preconditions.
- ☐ **F2** Split 1.2 into Log In / Log Out; split or per-operation-prioritise 6.3 so Must-shortlist and Should-notes are independently traceable.
- ☐ **F3** Reclassify 6.1 as an include-only sub-function: no primary actor, inherited priority, compressed internal algorithm.
- ☐ **F4** Add `Trigger` / `Extends` / `Included by` template rows; record 3.2→3.1 «extend» in a field; reclassify 8.2→8.1 as a precondition, not an include; reconcile 6.1’s caller list.
- ☐ **F5** Rewrite main flows as single branch-free success paths; move inline conditionals (1.3 s3, 4.2 s3.1) to extensions stated once; give 3.1 a definite typical scenario.

- ☐ **F6** Make every alternative-flow ID unique (AF-S3.1/.2/.3).
- ☐ **F7** Remove mechanism (hashed password, server-side token, request-scoped state) from flows; rely on the already-cited NFR/constraint.
- ☐ **F8** Reconcile the 19-vs-20 use-case count with the diagram; annotate FR-RES-02/03 as intentional cross-cutting requirements.

Ordering rationale: F1–F3 change the meaning of the contracts (what is guaranteed, what the goal is); F4–F5 fix how relationships and scenarios are expressed; F6–F8 are precision and bookkeeping. Items F1, F2, and F6 are small edits with high grader-visible payoff.

Appendix A — Traceability Matrix

Traceability Matrix — Requirements ↔ Use Cases ↔ Diagram

Backs the *Complete* (c) and *Traceable* (h) scores in `SRS_Review_IEEE830.md`. **Legend:** UC = use case in `Use Case Descriptions.md`; **Dgm** = present in `FamPlan_Use_Case_Diagram.drawio.png` (✔ yes / — n/a).

Functional requirements → use case → diagram

FR ID	Requirement (short)	UC	Dgm	Note
FR-ACC-01	Register account	1.1	✓	
FR-ACC-02	Authenticate	1.2	✓	login half
FR-ACC-03	Logout	1.2	✓	logout half
FR-ACC-04	Password reset	1.3	✓	
FR-ACC-05	Update account details	1.4	✓	editable-field list is a TBD (C1)
FR-ACC-06	Delete account	1.5	✓	
FR-PREF-01	Budget range	2.1	✓	6 FRs fold into 1 UC
FR-PREF-02	Towns/flat types/thresholds	2.1	✓	
FR-PREF-03	Priority destinations	2.1	✓	uses OneMap geocoding
FR-PREF-04	Amenity + barrier-free prefs	2.1	✓	
FR-PREF-05	Criterion weights 0–5	2.1	✓	
FR-PREF-06	Retrieve/update prefs	2.1	✓	
FR-SRCH-01	Search by town/street/block	3.1	✓	Actor “User” — see C4
FR-SRCH-02	Filter results	3.1	✓	
FR-SRCH-03	Apply saved preferences	3.2	✓	extends 3.1 (missing on diagram)
FR-SRCH-04	Label as profile not listing	3.1	✓	
FR-SRCH-05	Sort results	3.1	✓	
FR-SRCH-06	Reject invalid criteria	3.1	✓	
FR-MKT-01	Block/flat attributes	4.1	✓	
FR-MKT-02	Comparable transactions	4.2	✓	
FR-MKT-03	Count/median/min/max	4.2	✓	
FR-MKT-04	12/36/60-month period	4.2	✓	
FR-MKT-05	Trend time series	4.2	✓	
FR-MKT-06	Source/coverage/date	4.2	✓	
FR-MKT-07	Indicative-evidence label	4.2	✓	
FR-MKT-08	No-data vs zero distinction	4.2	✓	“insufficient” threshold is a TBD (B3)
FR-LOC-01	Block on map	5.1	✓	
FR-LOC-02	Nearby MRT/bus/school/amenity	5.1	✓	OneMap + LTA + geocoded schools
FR-LOC-03	500m/1km/2km radius	5.1	✓	
FR-LOC-04	Distance to each place	5.1	✓	
FR-LOC-05	Route + travel time	5.2	✓	

FR ID	Requirement (short)	UC	Dgm	Note
FR-LOC-06	Barrier-free routing/coverage	5.2	✓	
FR-LOC-07	Provider + retrieval time	5.1 / 5.2	✓	
FR-DEC-01	Suitability score	6.1	✓	include-only sub-function (F4)
FR-DEC-02	Score breakdown	6.1	✓	“documented rounding” TBD (B3)
FR-DEC-03	Missing-data treatment	6.1	✓	
FR-DEC-04	Compare 2–4 profiles	6.2	✓	
FR-DEC-05	Add to shortlist	6.3	✓	
FR-DEC-06	Remove from shortlist	6.3	✓	
FR-DEC-07	Private notes	6.3	✓	length boundary is a TBD (B3)
FR-HFE-01...10	Indicative HFE pre-check	7.1	✓	10 FRs fold into 1 UC
FR-ADM-01	Require admin role	8.1 / 8.2	✓	gating precondition
FR-ADM-02	Source status view	8.1	✓	
FR-ADM-03	Manual refresh	8.2	✓	
FR-ADM-04	24-hour scheduled check	8.3	✓	Actor “System clock” vs diagram label
FR-ADM-05	Sync history	8.1	✓	folded into 8.1’s flow
FR-RES-01	Preserve last valid dataset	8.3	✓	
FR-RES-02	Fault isolation	—	—	cross-cutting; verified by NFR-REL-03 (C3)
FR-RES-03	Stale-data warning	—	—	cross-cutting; verified by NFR-DATA-02 (C3)

Forward coverage: every FR except the two cross-cutting resilience items (FR-RES-02/03) maps to a use case, and every use case appears on the diagram. No orphan use cases.

Non-functional requirements → binding requirement/use case

NFRs are cross-cutting; they trace to the FRs/UCs they constrain rather than to a use case of their own.

NFR	Binds to
NFR-PERF-01 / SCALE-01	FR-SRCH-01/02 (UC 3.1)
NFR-PERF-02	FR-DEC-01/04 (UC 6.1/6.2)
NFR-PERF-03	FR-LOC-05 (UC 5.2)
NFR-USAB-01	UC 3.1 → 6.2 flow
NFR-USAB-02	every validation-bearing FR
NFR-ACC-01/02	core workflows (UC 1.x, 3.1, 4.x, 6.x, 7.1)
NFR-SEC-01...05	FR-ACC-02 (UC 1.2), FR-ADM-01 (UC 8.1), all input
NFR-DATA-01...03	every externally sourced output (UC 4.x, 5.x, 8.x)
NFR-PRIV-01	FR-HFE-* (UC 7.1)
NFR-PRIV-02	FR-ACC-06 (UC 1.5)
NFR-PRIV-03	FR-ACC-01 (UC 1.1), FR-PREF-* (UC 2.1)
NFR-MAINT-01/02, TRANS-01/02, REL-01...03	scoring, HFE, sync, resilience clusters

Data dictionary → requirements (spot check of backward traceability)

Every Data Dictionary entity cites the FR(s) it serves (e.g. *Suitability Score* → FR-DEC-01; *Sync History Record* → FR-ADM-05; *HFE Session Input* → FR-HFE-02). Backward traceability from data model to requirements is **complete** — a genuine strength. The one correction needed is the actor definition in Data Dictionary §8 (see C4).

Appendix B — Recommended Canonical Glossary

FamPlan — Recommended Canonical Glossary

Produced by the domain-modeling pass of this review. This is a **glossary only** — no implementation detail. It records the terms that were *contested or fuzzy* across the Lab 1 artifacts and the single canonical form recommended for each. Items marked $\triangle$ **decision pending** are recommendations the team should ratify, then propagate to the requirements doc, use case descriptions, data dictionary, and diagram so all four agree.

Adopt by moving this file to the repo root (or `Lab1/`) once ratified.

Actors

Visitor $\triangle$ decision pending (naming)

An unauthenticated member of the public. May search, view block-and-flat profiles and market history, view nearby places and routes, compare profiles, and run the indicative HFE pre-check. Cannot save anything.

Resolves the conflict: the requirements and diagram say **Visitor**; the Data Dictionary §8 calls the same actor “Guest / Applicant”. Use **Visitor** everywhere; retire “Guest” and “Applicant”.

Registered User

A Visitor who has created an account (FR-ACC-01). Everything a Visitor can do, **plus** saved preferences and weights, a shortlist, and private notes. **Correction required:** Data Dictionary §8 currently defines Registered User as “the actor behind every FR whose Actor column reads ... ‘User’.” That wrongly excludes Visitors from `Actor = User` requirements. A Registered User is **not** the actor for `User` requirements — see **User** below.

Data Administrator

A Registered User holding the administrator role (FR-ADM-01). Everything a Registered User can do, plus source sync status, manual refresh, and sync history.

User (abstract) $\triangle$ decision pending (scope)

The abstract superset **Visitor OR Registered User**. Any requirement whose Actor column reads “User” is available to **both** — including unauthenticated Visitors. This is §4’s meaning and the one recommended. (This is the single most important term to ratify: it decides whether a Visitor can search, view market data, compare, and run the HFE pre-check. Recommended answer: **yes**.)

System (scheduled process) $\triangle$ decision pending (naming)

Scheduled processes acting with no user present (FR-ADM-04, FR-RES-01). **Resolves the conflict:** FR-ADM-04’s Actor column says “**System clock**”; the diagram and UC 8.3 say “**System (scheduled process)**”. Use one label — recommended **System (scheduled process)**.

External Data Provider (supporting actor)

A named source the application does not own: **data.gov.sg** (HDB Resale, HDB Property, MOE Schools), **OneMap** (SLA — geocoding, routing, barrier-free routing, nearby-place themes), **LTA DataMall** (MRT/LRT station coordinates). A secondary actor, not a user role. Currently absent from the use case diagram (see `Diagram_Critique.md` F3).

Core domain terms (already well-defined — recorded to prevent drift)

Block-and-Flat Profile

One combination of block + street + flat type the app can show market history and a score for. **Deliberately not** “Listing” or “Unit”: it is assembled from historical transactions and is **not** a live for-sale listing (FR-SRCH-04). Keep the word “Profile” in both UI and code.

Suitability Score

The single computed number for how well a Block-and-Flat Profile matches a user’s weights. Keep three concepts distinct — they are easy to conflate: - **Criterion Weight** — the *user’s* personal 0–5 importance per criterion. - **Scoring Configuration** — the *system-wide* versioned normalisation constants. - **Suitability Score** — the *output* computed from a profile + weights + configuration.

Indicative (market evidence / HFE pre-check)

A required framing word, not decoration. Every market figure is **indicative market evidence**, never an official valuation or price prediction (FR-MKT-07); every HFE output is **indicative guidance**, never an official assessment (FR-HFE-01/08). “Indicative” must appear in the output payload, not only in interface copy (NFR-DATA-01, NFR-TRANS-02).

Data Availability State

The three-way distinction **no matching transactions** / **insufficient data** / **source unavailable**, which must replace a bare `0` or `null` (FR-MKT-08). “Missing” is never “zero”.

Open vocabulary items (unset values, from SRS review B3/C1)

These terms are *used* in the spec but their values are undefined — resolve and record here: shortlist **size limit**; note **length boundary**; suitability-score **rounding rule**; market “**insufficient data**” **threshold**; FR-ACC-05 **editable-field list**.